CompletableFuture and Asynchronous Composition

9 min read Updated 28 Jul 2026 Topic 42 of 65

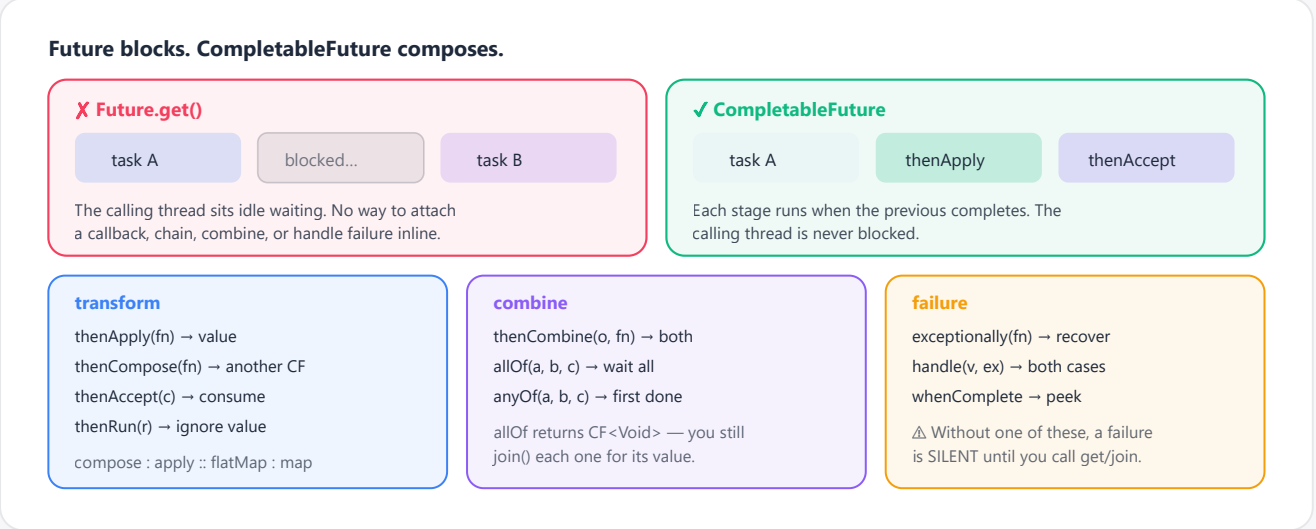
Bookmark Mark done

Why it matters

`Future` was Java 5's answer to asynchronous work, and it is barely usable: the only way to get the result is `get()`, which blocks. You cannot chain, combine, or handle failure without blocking first. `CompletableFuture` fixes all of it, and it is the standard way modern Java composes asynchronous calls — which makes "call three services and combine the results" a very common interview exercise.

The concept, simply

A `CompletableFuture<T>` is a promise of a value that will exist later, with methods to describe what should happen when it does — without blocking.



The API divides cleanly into three groups: transform one result, combine several, and handle failure. Learning which group you need is most of the battle.

How it works

should almost always pass one, because the default is the shared `ForkJoinPool.commonPool()` with all the problems from Chapter 7.

The naming convention is regular, which makes the large API learnable:

- `thenApply` — transform the value (like `map`).
- `thenCompose` — the function returns another future (like `flatMap`). Prevents `CompletableFuture<CompletableFuture<T>>`.
- `thenAccept` — consume the value, return nothing.
- `thenRun` — run something, ignoring the value.
- `thenCombine` — merge two independent futures.
- The `Async` **suffix** on any of these runs the callback on a different thread rather than the one that completed the previous stage.

Failure handling is where `CompletableFuture` differs most from synchronous code: an exception does not propagate — it completes the future *exceptionally*, and every downstream stage is skipped until something handles it. If nothing does and nobody calls `join()`, the failure is completely silent.

Code examples

JAVA

Copy

```java
// ⚠ Always pass an executor — the default is the shared common pool.
ExecutorService io = Executors.newFixedThreadPool(20);

CompletableFuture<String> f = CompletableFuture
    .supplyAsync(() -> fetchUser(id), io)
    .thenApply(User::email)                          // transform
    .thenApply(String::toLowerCase);

// thenCompose when the function returns ANOTHER future — the flatMap case
CompletableFuture<Order> order = CompletableFuture
    .supplyAsync(() -> fetchUser(id), io)
```

Combining, which is the classic interview task — three calls in parallel:

JAVA Copy

```
CompletableFuture<Profile> profile = supplyAsync(() -> fetchProfile(id), i
CompletableFuture<List<Order>> orders = supplyAsync(() -> fetchOrders(id),
CompletableFuture<Prefs> prefs = supplyAsync(() -> fetchPrefs(id), io);

// All three run concurrently; total latency ≈ the SLOWEST, not the sum.
CompletableFuture<Dashboard> dashboard =
    CompletableFuture.allOf(profile, orders, prefs)
        .thenApply(v -> new Dashboard(
            profile.join(),      // ✓ safe — allOf guarantees completion
            orders.join(),
            prefs.join()));

// ⚠ allOf returns CompletableFuture<Void> — it does NOT collect results.
//   You still join() each one, which is safe only after allOf completes.

// Two futures with a combining function:
CompletableFuture<String> both = profile.thenCombine(prefs,
        (p, pr) -> p.name() + " likes " + pr.theme());

// First one to finish wins — useful for hedged requests:
CompletableFuture<Object> fastest = CompletableFuture.anyOf(replicaA, repl
```

Failure handling, including the timeout everyone forgets:

JAVA Copy

```
CompletableFuture<String> safe = supplyAsync(this::riskyCall, io)
    .orTimeout(2, TimeUnit.SECONDS)              // Java 9+ — fail if slo
    .exceptionally(ex -> {                       // recover with a fallba
        log.warn("call failed, using cache", ex);
        return cache.get(id);
    });
```

```java
// handle() sees BOTH outcomes in one place:
CompletableFuture<String> h = supplyAsync(this::riskyCall, io)
    .handle((value, ex) -> ex != null ? "fallback" : value.toUpperCase());

// whenComplete() observes without changing the result — good for logging:
future.whenComplete((v, ex) -> {
    if (ex != null) log.error("failed", ex); else log.info("ok: {}", v);
});

// ⚠ The exception you catch is wrapped in CompletionException:
future.exceptionally(ex -> {
    Throwable cause = ex instanceof CompletionException ? ex.getCause() :
    return handleByType(cause);
});

// ⚠ SILENT FAILURE: with no exceptionally/handle and no join(), a failed
//   future is simply discarded. Nothing is logged. Always terminate a cha
//   with failure handling.
supplyAsync(this::riskyCall, io).thenAccept(this::use);   // ✗ swallows ex
```

`join()` versus `get()`:

JAVA Copy

```java
future.get();     // throws CHECKED InterruptedException + ExecutionExcept
future.join();    // throws UNCHECKED CompletionException — usable in lamb
// join() is the one you want inside stream pipelines and callbacks.
```

Under the hood

The default executor is a real hazard. Without an explicit `Executor`, async stages run on `ForkJoinPool.commonPool()` — shared JVM-wide, sized to cores minus one, and designed for short CPU-bound work. Blocking HTTP calls there starve every other user of that pool, including parallel streams in library code. Passing your own executor is not a refinement; it is the correct default.

Which thread runs a callback is subtler than it looks. A non-`Async` method such as `thenApply` may run on the thread that completed the previous stage — or on the

explicit executor when it matters, and never assume a callback is on a particular thread.

Ordering pitfall: `supplyAsync(a).thenApply(b)` runs a and b *sequentially*. Only independent futures created up front run concurrently. A very common mistake is chaining three service calls with `thenCompose` and expecting parallelism — they are a pipeline, and the latency is the sum.

Java 21 alternative: with virtual threads, plain blocking code inside a structured-concurrency scope achieves the same fan-out with far less ceremony, because blocking is cheap. `CompletableFuture` remains the standard for now and for pre-21 codebases, but "how would you write this on Java 21?" is an increasingly common follow-up.

Common mistakes

- **Not passing an executor.** Blocking work lands on the shared common pool.
- `thenApply` **where** `thenCompose` **is needed.** Produces a nested future.
- **No failure handling.** A failed future with no `join()` fails completely silently.
- **Expecting** `allOf` **to return results.** It returns `Void`; join each future afterwards.
- **Calling** `join()` **too early.** That blocks and discards the point of the API.
- **Chaining and expecting parallelism.** A chain is sequential; create futures up front to run concurrently.
- **Forgetting the** `CompletionException` **wrapper.** Unwrap with `getCause()`.
- **No timeout.** A hanging dependency hangs the chain forever — use `orTimeout`.

Interview lens

> Q: "`Future` vs `CompletableFuture`?"
> A: `Future` only lets you poll or block on `get()` — no chaining, no combining, no callbacks, no failure handling without blocking first. `CompletableFuture` implements `CompletionStage`, so you can describe what happens when the value

arrives: transform it, compose further async calls, combine several futures, handle exceptions, and apply timeouts — all without blocking a thread.

Q: "`thenApply` or `thenCompose`?"

A: `thenApply` when the function returns a plain value; `thenCompose` when it returns another `CompletableFuture`. It is exactly `map` versus `flatMap` — using `thenApply` for an async call gives you a `CompletableFuture<CompletableFuture<T>>` that you then have to unwrap.

Q: "Call three services in parallel and combine the results."

A: Create all three futures with `supplyAsync` and an explicit executor so they start immediately, then `allOf(a, b, c).thenApply(v -> combine(a.join(), b.join(), c.join()))`. Total latency is the slowest call rather than the sum. The detail people miss is that `allOf` returns `CompletableFuture<Void>` — it signals completion but does not collect results, so you still join each one, which is safe at that point.

Q: "Which thread runs the callbacks?"

A: Without an explicit executor, the common `ForkJoinPool` — shared across the JVM and sized for CPU work, so blocking calls there starve everything else. Non-`Async` methods may also run on whichever thread completed the previous stage, or even the caller if the future was already complete. So I always pass my own executor for anything blocking and never assume which thread a callback is on.

Q: "How do you handle failures?"

A: `exceptionally` to recover with a fallback, `handle` to deal with success and failure in one place, or `whenComplete` to observe without altering the result. The critical thing is that an exception completes the future exceptionally rather than propagating — downstream stages are skipped, and if nothing handles it and

nobody joins, the failure is completely silent. I also add `orTimeout`, because otherwise a hung dependency hangs the chain forever.

Q: "`get()` or `join()`?"
A: Both block; the difference is the exceptions. `get()` throws checked `InterruptedException` and `ExecutionException`; `join()` throws unchecked `CompletionException`, which makes it usable inside lambdas and streams where checked exceptions are not allowed. Either way, calling one early defeats the purpose — the value of the API is not blocking.

Cheat-card

- `supplyAsync` / `runAsync` to start — **always pass an** `Executor` (default is the shared common pool).
- `thenApply` = map · `thenCompose` **= flatMap** · `thenAccept` / `thenRun` = consume.
- `thenCombine` (two) · `allOf` (all, returns `Void`) · `anyOf` (first).
- `exceptionally` · `handle` · `whenComplete` · `orTimeout`.
- **Unhandled failure with no join = completely silent.**
- Exceptions arrive wrapped in `CompletionException` — unwrap with `getCause()`.
- **A chain is sequential**; create futures up front for parallelism.
- `join()` = unchecked (lambda-friendly) · `get()` = checked.
- Java 21: virtual threads + structured concurrency do this with plain blocking code.

QUICK CHECK — no marks, just for you.

1. When should you use thenCompose rather than thenApply?

- ○ When the function itself returns another CompletableFuture — otherwise you get CompletableFuture<CompletableFuture<T>>
- ○ When you need the result on a different thread

They are interchangeable

When the operation is synchronous

Check answer

2. What does CompletableFuture.allOf(a, b, c) return?

The first result to complete

A CompletableFuture<Void> — it signals completion but does NOT collect results, so you still join each future afterwards

A blocking iterator over the results

A CompletableFuture<List<T>> containing all three results

Check answer

3. Why should you always pass an Executor to supplyAsync?

Because otherwise the future never completes

Because supplyAsync requires it

Because the default is the shared ForkJoinPool.commonPool(), sized for short CPU-bound work — blocking calls there starve every other user of that pool across the whole JVM

Because the default executor is single-threaded

Check answer

Executors and Thread Pools
Next
Atomics, CAS and Lock-Free Concurrency

LEARN MORE

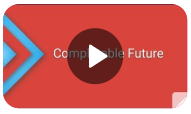
ARTICLES & DOCS

- CompletableFuture — API docs
  docs.oracle.com
- CompletionStage — API docs
  docs.oracle.com

VIDEOS



- Java 8 CompletableFuture Tutorial with Examples | runAsync() & supplyAsync() | JavaTechie | Part 1
  Java Techie



- Introduction to CompletableFuture in Java 8
  Defog Tech

Hand-picked references for this lesson.